Governance, Authority, and Policy Reconciliation in Agentic IDE Ecosystems: A Comprehensive Analysis of the Antigravity Framework and Multi-Agent Orchestration

The transition from traditional, human-centric Integrated Development Environments (IDEs) to autonomous, agentic platforms marks a paradigm shift in software engineering comparable to the move from assembly to high-level programming languages. At the center of this evolution is Google Antigravity, an agent-first development platform released in late 2025 that leverages the reasoning capabilities of the Gemini 3 Pro model to plan, execute, and verify complex software projects.[1] However, as these systems gain the ability to perform multi-step workflows and reason across entire repositories, they introduce a novel class of organizational friction known as the "Authority Conflict".[3] This conflict typically manifests when a high-level Decision Processor (DP)—the system component responsible for task delegation and strategic planning—issues directives that clash with the repository's standing governance rules, such as those codified in AGENTS.md or specialized task checklists like TASK.md.[5]

The emergence of these conflicts highlights the fundamental tension between autonomous goal-seeking behavior and the safety guardrails necessary for secure, professional development. For the worker agent, identifying such a conflict is not merely a technical error but a critical governance failure that requires immediate reconciliation. When a DP instructs a worker to stage and commit changes while repository-level instructions explicitly prohibit git actions by agents, the system reaches a state of operational deadlock. Resolving this requires a sophisticated understanding of the hierarchical relationship between the DP, the operator, the governance substrate, and the various state-tracking artifacts that define the agent's lifecycle.[5]

The Theoretical Foundations of Agentic Authority

The concept of authority conflict in agentic systems is deeply rooted in 20th-century organizational theory. Scholars such as Max Weber identified that specialization within a bureaucracy is most effective when tasks are repeatable and similar, yet they cautioned that such specialization is constrained by the organizational scholar's ability to provide context and define authority boundaries.[3] In modern agentic frameworks, the "worker" agent is the specialized entity, while the DP serves as the hierarchical superior, reflecting the Military Decision-Making Process (MDMP) where a commander (or supervisor) provides intent and

guidance to a staff charged with executing operation plans.[3]

The MDMP is a systematic process designed to solve problems and establish conditions for effective decisions under time-constrained, high-tempo conditions.[7] When this logic is applied to Google Antigravity, the DP acts as the orchestrator of the "Ultrawork Loop"—a cycle of planning, acting, verifying, and fixing.[8] The conflict arises because the DP's logic is often dynamic and goal-oriented, whereas the governance rules in AGENTS.md are static and constraint-oriented.[2] This divergence between "intent" (what must be done) and "policy" (how it must not be done) creates a "governance void," a state where formal mechanisms fail to clarify authority relations or resolve conflicts of interest.[9]

Authority Component	Role in the Ecosystem	Primary Artifact	Reasoning Logic
Decision Processor (DP)	High-level strategy and task delegation.	implementation_plan.md	Goal-seeking; prioritizing task completion and state progression.[4]
Worker Agent	Execution of granular task clusters.	TASK.md (Checklist)	Script-driven; following explicit steps and success criteria.[6]
Governance Layer	Guardrails and operational constraints.	AGENTS.md	Rule-based; prioritizing security, safety, and compliance.[5]
Operator (Human)	Final authority and conflict arbiter.	Interactive Console	Decision-based; weighing risk vs. benefit and authorizing overrides.[1]

Governance Substrates: The Role of AGENTS.md and.agent/rules/

In the Antigravity and GitHub Copilot ecosystems, governance is defined through "instruction substrates" that the IDE auto-includes in every agent request.[5] The AGENTS.md file serves as the single source of truth, moving the AI from a generic assistant to a team of virtual

specialists with clear roles, commands, and boundaries.[5] This file is treated as a policy rather than documentation, specifying technical context, project structures, and critical limits such as the prohibition of git actions or the protection of production secrets.[5]

Security Implications of Policy-Based Control

The reliance on markdown-based policy files introduces a vulnerability known as "Agent Goal Hijacking".[10] Because the IDE search is workspace-aware and enabled by default, a malicious contributor could inject instructions into an AGENTS.md file that convince the agent to exfiltrate data or sabotage a project during an everyday coding session.[10] This security risk explains why many sophisticated repositories prohibit autonomous git actions in their AGENTS.md files; by preventing an agent from staging and committing, the organization ensures that a human must review all changes before they are persisted to the version history.[5]

This "Soft Security" model is vulnerable to prompt injection, where a goal defined by the DP might inadvertently (or maliciously) override the safety policy.[11] The conflict mentioned in the user query—where the DP instructs the worker to commit despite a prohibition—is a direct manifestation of this structural vulnerability. The agent's refusal to proceed without an override is a vital safety check, preventing the violation of the repository's security posture.[3]

Comparative Governance in Agentic IDEs

Different platforms implement governance through various directories and triggers. While GitHub Copilot and VS Code Chat rely heavily on AGENTS.md, Google Antigravity distinguishes between passive rules and active workflows.[2]

- **Rules (.agent/rules/):** These are passive constraints that are always "on" or triggered by specific file types. They restrict the *how* of every task and serve as the system's guardrails.[2]
- **Workflows (.agent/workflows/):** These are active, user-triggered macros (e.g., /test or /review) that allow for manual orchestration of complex steps.[2]
- **Skills (.agent/skills/):** These are on-demand capability extensions that provide specific instructions and scripts only when a relevant task is active.[2]

This distinction is vital for proper system design, as it allows the agent to maintain a high degree of specialization without saturating its context window with irrelevant instructions.[2]

The Lifecycle of a Task: TASK.md and implementation_plan.md

If AGENTS.md is the repository's constitution, then the task management files are its executive orders. In Antigravity, the process of turning an idea into execution follows a

three-part lifecycle involving the Blueprint, the Checklist, and the Proof.[6]

The Blueprint: implementation_plan.md

Before the agent writes a single line of code, it generates the implementation_plan.md. This is a technical design document proposed by the agent and reviewed by the human architect.[6] The value of this blueprint lies in its ability to catch architectural misunderstandings early; catching an error in the plan is cheap, while catching it in the code is expensive.[6] The plan outlines how the agent intends to solve a problem, such as using a byte-based cost model for a database optimizer rather than a simple row count.[6]

The Checklist: TASK.md

Once the plan is approved, it is decomposed into the TASK.md (or task.md), a dynamic to-do list that tracks dependencies and granular, executable steps.[6] In multi-agent environments like the Antigravity Swarm, this file becomes part of the "Manus Protocol," serving as the shared memory for the agent team.[8] The orchestrator reads and writes to this file in real-time, checking off items as they are completed. This allows the human operator to monitor progress without constantly reading terminal output.[6]

The Proof: walkthrough.md

The final artifact is the walkthrough.md, which serves as a verification guide and a "signed receipt" for the completed task.[6] This file details exactly how the agent validated its changes, including the CLI commands it ran and the test outputs it observed.[6] This artifact is essential for the "trust but verify" model of agentic development, allowing the human to replicate the agent's verification steps locally.[6]

Artifact	Phase	Ownership	Primary Function
implementation_plan.md	Planning	Agent (Proposed) / Human (Approved)	Architectural design and design-doc validation.[6]
TASK.md	Execution	Agent (Orchestrated)	Real-time state tracking and dependency management.[6]
walkthrough.md	Verification	Agent (Generated)	Audit trace and

			verification manual for the operator.[6]

Dissecting the Conflict: The Worker vs. the Decision Processor

The specific authority conflict presented occurs when the DP—acting as the superior authority—instructs the worker to bypass the restrictions codified in TASK.md and AGENTS.md. The worker's response is a model of disciplined initiative, recognizing that it cannot violate the standing governance rules without an explicit override from the Operator.[1]

The 10 Out-of-Scope Changes

A critical detail in the user query is the existence of "10 out-of-scope changes." In the context of agentic development, scope control is maintained through an "allowlist" of files and directories that the agent is permitted to modify.[5] When an agent identifies changes that fall outside this allowlist, it must halt to prevent unintended side effects or security breaches.[13] This is particularly important in environments without comprehensive test frameworks, where manual code review is the primary validation method.[14]

The worker offers four distinct paths for resolution, each reflecting a different level of trust and operational risk.[1] These options are consistent with the "Agent-driven" vs. "Review-driven" modes of Google Antigravity, where the operator "drives the car" by providing high-level instructions while the AI handles the planning and execution.[1]

Path 1: Scope Isolation and Cleaning

The first option—cleaning or isolating the 10 out-of-scope changes—is a defensive strategy. By removing the problematic changes, the operator allows the agent to continue its mission within the safely defined boundaries of the repository.[14] This approach adheres to the "Non-Destructive Modifications" principle, which mandates extreme caution when modifying code that lacks automated verification.[14] Isolation ensures that the agent's actions remain "workspace-aware" but restricted to active URIs.[11]

Path 2: Allowlist Expansion

The second option involves expanding the allowlist to include the previously out-of-scope files. This is a "Policy Adaptation" approach. In the Antigravity ecosystem, skills and rules are "on-demand," and updating the AGENTS.md or task-specific constraints allows the agent to legally perform the work requested by the DP.[5] By providing this specificity, the operator reduces ambiguity and allows the agent to generate code tailored to the exact project environment.[5]

Path 3: Operator Handover (The Manual Commit)

The third option—where the operator performs the commit and the agent only applies the TASK.md and SoP.md edits—is a "Human-in-the-Loop" (HITL) resolution. This is often the preferred method for "experimental tier" engineering, where the human architect handles the boilerplate of version control while the agent fills in the details.[6] This path recognizes the "Validator Rule," where the human remains the final authority on the quality and integrity of the repository state.[8]

Path 4: Governance Override

The fourth option—an explicit governance override—is the most aggressive resolution. It essentially tells the agent: "I am aware of the rule in AGENTS.md, and I am authorizing you to break it for this specific task." In organizational science, this is known as "lateral authority," where merit-based decisions derived from competence and conflict resolution override centralized, formal mechanisms.[9] This override must be documented in the SoP.md recovery log to ensure an audit trace exists for why the governance boundary was breached.[6]

Recovery Logs and Standard Operating Procedures: The Role of SoP.md

The SoP.md (Standard Operating Procedure) file is an AI-powered assistant implemented as an Agent SOP—a markdown-based standard for encoding AI agent workflows.[15] It utilizes RFC 2119 constraints (MUST, SHOULD, MAY) to ensure reliable execution while preserving the agent's ability to reason through complex scenarios.[15]

The Recovery Log

In the user's scenario, the SoP.md is used to maintain a "recovery log." This is an essential component of "Reliability-Focused Approaches" used in settings affected by crisis or conflict.[16] Just as field-implementers in fractured authorities must negotiate uncertainties daily to deliver assistance reliably, AI agents must use recovery logs to navigate state inconsistencies caused by authority conflicts.[16]

The recovery log in SoP.md serves several functions:

1. **State Reconstruction:** It records the agent's reasoning during the conflict, allowing it to resume work if the session is interrupted or if the operator provides an override.[8]
2. **Anomaly Identification:** It analyzes logs to identify patterns and anomalies, ensuring server (or agent) reliability and performance.[17]
3. **Governance Auditing:** It provides an immutable log of what has been done, which is crucial for missions involving parallel agent orchestration where multiple agents might otherwise create race conditions.[8]

RFC 2119 in Agent Governance

The use of MUST, SHOULD, and MAY in SoP.md allows the repository owner to define the "strength" of governance rules. A rule stating "The agent MUST NOT commit to the main branch" is a hard constraint, while "The agent SHOULD run linting before staging" is a recommendation that can be bypassed in an emergency.[15] When the DP instructs a commit that violates a MUST constraint, the worker is duty-bound by its SOP to halt and request an override.[5]

Multi-Agent Orchestration and the Manus Protocol

When using the Antigravity Swarm framework, authority conflicts become even more complex because they involve multiple specialized agents: Architects, Engineers, and Validators.[8] These agents collaborate through the "Manus Protocol," a rigorous state management system that uses persistent markdown files in the project root.[8]

Parallel Execution and Race Conditions

The primary advantage of a swarm is parallel execution—why wait for one agent when you can have one writing docs, one writing tests, and one writing code simultaneously?.[8] However, this parallelism creates a risk of race conditions if multiple agents attempt to update the task_plan.md or findings.md at once.[8] The Manus Protocol mitigates this by enforcing a "Validator" rule: the final agent in any generated team is strictly a Quality Validator tasked with ensuring that the task_plan.md is complete and that the user's requirements are met.[8]

Shared Memory and Findings.md

In a swarm, the findings.md file serves as a "shared scratchpad" where agents exchange knowledge.[8] If an Engineer agent encounters an authority conflict, it records this in findings.md, allowing the Architect agent to potentially refactor the plan or the Validator agent to flag the issue to the Operator.[8] This externalization of the thought process is what transforms the AI from a code completion tool into a disciplined collaborator.[6]

Swarm Role	Responsibility	Interaction with Governance
Architect	Strategic planning and blueprint generation.	Translates DP intent into a safe implementation_plan.md.[6]

Engineer	Execution of code changes and script running.	Operates within the AGENTS.md allowlist; identifies conflicts.[5]
Validator	Quality assurance and final verification.	Strictly enforces the "Validator Rule"; checks task_plan.md completeness.[8]
Librarian	Knowledge retrieval and context management.	Searches findings.md and AGENTS.md to ground agent actions.[8]

Command Resolution and mdflow: Executable Markdown

A unique aspect of task execution in modern agentic setups is "mdflow," which transforms markdown files into executable commands.[18] This system uses filename-based command resolution to route tasks to the appropriate model or provider, such as task.claude.md for Claude Code or task.copilot.md for GitHub Copilot.[18]

Executable Markdown Patterns

In mdflow, the TASK.md file is not just a static checklist but a runnable script.[18] It supports inline command execution using shell patterns (e.g., ! git log --oneline). This capability is why the "No Git" prohibition in AGENTS.md is so critical; if an agent can execute arbitrary shell commands via its task file, it has the power to modify the entire repository unless explicitly sandboxed.[2]

mdflow provides core flags for sandboxing and permissions:

- --sandbox read-only: Prevents any file modifications.[18]
- --sandbox workspace-write: Allows edits within the project folder.[18]
- --ask-for-approval: Defines when the agent should pause for operator intervention (e.g., untrusted, on-failure, never).[18]
- --yolo: Bypasses all approvals and sandboxing—essentially a permanent governance override.[18]

This level of granularity allows the operator to tailor the agent's autonomy to the specific needs of the project, though "full-auto" modes are generally discouraged due to the risk of "context rot" and goal hijacking.[10]

Professional Reporting and the Walkthrough Receipt

In professional engineering environments, every task completion must be accompanied by an audit trace.[6] In the Antigravity framework, this is achieved through "Receipts" and "Replays".[6] The walkthrough.md acts as a signed receipt, detailing exactly how the agent validated its own changes.[6]

Audit Trace Mechanics

The audit trace is crucial because it facilitates "trust but verify".[6] When an agent claims it has fixed a bug or implemented a feature, the operator doesn't have to take its word for it. The walkthrough.md lists:

1. The specific CLI commands the agent ran (e.g., npm run test:auth).[6]
2. The test outputs it observed (e.g., "All 5 tests passed").[6]
3. A sample snippet or CLI command for the operator to verify the work (e.g., cargo run with a sample SQL query).[6]

For the user in the query, who has already dealt with the TASK.md and SoP.md edits manually, the agent's next step is to generate these receipt changes. The "TASK.md receipt changes" refer to updating the checklist to reflect the final state of the task, while the "SoP.md recovery log" ensures that the history of the authority conflict and its resolution is preserved for future architectural reviews.[6]

Mathematical Modeling of Agentic Reliability

The reliability of an agentic workflow (R) can be modeled as a function of task granularity (G), governance density (D), and operator intervention frequency (I):

$$R = \frac{G \cdot I}{D}$$

Where:

- **Granularity (G):** Breaking down complex issues into focused sub-issues increases high-quality output and makes pull requests easier to understand.[13]
- **Intervention (I):** Frequent checkpoints and plan reviews prevent architectural misunderstandings.[1]
- **Density (D):** Excessive or contradictory governance rules (like a DP instruction vs. an AGENTS.md prohibition) can decrease reliability by creating operational deadlocks.[3]

To maximize R, the system must aim for well-scoped issues with clear acceptance criteria and custom instructions that keep prompts concise.[13] When an authority conflict arises, D increases, requiring a corresponding increase in I (Operator intervention) to restore R.

Synthesis of Findings and Future Outlook

The conflict between the Decision Processor and repository governance is an inevitable byproduct of the shift toward autonomous, multi-agent software engineering. The Antigravity framework, with its structured lifecycle of blueprints, checklists, and proofs, provides a robust template for managing these conflicts, but it remains dependent on the "Soft Security" of markdown-based policies.

Core Principles for Conflict Resolution

Based on the analysis of the Antigravity and Manus protocols, several core principles emerge for professional peers managing agentic repositories:

1. **Strict Hierarchy of Instruction:** The Operator's real-time, explicit command must always be the final authority, capable of overriding both the DP and the static rules in AGENTS.md.[1]
2. **Persistence of State:** Use artifacts like TASK.md and progress.md as "shared memory" to ensure that parallel agents remain synchronized and that the history of the mission is preserved.[8]
3. **The Validator Requirement:** Never deploy an autonomous swarm without a mandatory Validator agent tasked with enforcing governance and checking the completeness of the task plan.[8]
4. **Auditability as a First-Class Citizen:** Every task, especially those involving policy overrides, MUST generate a receipt (walkthrough.md) and a recovery log (SoP.md) to facilitate auditing and long-term knowledge management.[6]

As AI models like Gemini 3 Pro continue to evolve, we can expect "Experimental Tier" engineering to become the standard. In this future, agents will handle the toolchain, the boilerplate, and the initial testing, while human engineers move into the role of high-level architects and "Governance Officers".[6] The ability to diagnose and repair mistakes by reading logs and recognizing policy violations will be the hallmark of the next generation of software engineering.[6] The authority conflict described by the worker agent is not a sign of a failing system, but of a maturing one—one that recognizes its own boundaries and seeks human guidance to ensure the integrity and security of the code it creates.

Works cited

1. How to Set Up and Use Google Antigravity - Codecademy, accessed February 3,

2026, https://www.codecademy.com/article/how-to-set-up-and-use-google-antigravity
2. How to Build Custom Skills in Google Antigravity: 5 Practical ..., accessed February 3, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d
3. masterarbeit / master's thesis - PHAIDRA, accessed February 3, 2026, https://phaidra.univie.ac.at/api/object//o:1872408/download
4. Autonomous-Agents/README.md at main - GitHub, accessed February 3, 2026, https://github.com/tmgthb/Autonomous-Agents/blob/main/README.md
5. What are Agents.md and what are they for? - Itequia, accessed February 3, 2026, https://itequia.com/agents-md-in-github-copilot-a-practical-guide-to-creating-specialized-agents/
6. From Idea to Execution: Building a BigQuery SQL Optimizer with ..., accessed February 3, 2026, https://medium.com/google-cloud/from-idea-to-execution-building-a-bigquery-sql-optimizer-with-antigravity-and-gemini-3-19d3ed280071
7. MILITARY DECISION-MAKING PROCESS - U.S. Army, accessed February 3, 2026, https://api.army.mil/e2/c/downloads/2023/11/17/f7177a3c/23-07-594-military-decision-making-process-nov-23-public.pdf
8. wjgoarxiv/antigravity-swarm: ::A team of AI agents to code ... - GitHub, accessed February 3, 2026, https://github.com/wjgoarxiv/antigravity-swarm
9. From Governance Void to Interactive Governing Behaviors in New Research Networks | Academy of Management Discoveries, accessed February 3, 2026, https://journals.aom.org/doi/10.5465/amd.2014.0103
10. VS Code AGENTS.MD: Hidden Agent Instructions and Data Risk - Prompt Security, accessed February 3, 2026, https://prompt.security/blog/when-your-repo-starts-talking-agents-md-and-agent-goal-hijack-in-vs-code-chat
11. Leaked Antigravity System Prompt Report | PDF | Command Line ..., accessed February 3, 2026, https://www.scribd.com/document/986523905/Leaked-Antigravity-System-Prompt-Report
12. Authoring Google Antigravity Skills, accessed February 3, 2026, https://codelabs.developers.google.com/getting-started-with-antigravity-skills
13. Getting better results from Copilot Coding Agent #174930 - GitHub, accessed February 3, 2026, https://github.com/orgs/community/discussions/174930
14. AGENTS.md - miantiao-me/cloud-code - GitHub, accessed February 3, 2026, https://github.com/miantiao-me/cloud-code/blob/master/AGENTS.md
15. Eval SOP - AI-Powered Evaluation Workflow - Strands Agents, accessed February 3, 2026, https://strandsagents.com/latest/documentation/docs/user-guide/evals-sdk/eval-sop/
16. Providing social assistance and humanitarian relief: The case for embracing

uncertainty, accessed February 3, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC9541537/
17. FREE AI Prompts Library | Sabrina Ramonov - Notion, accessed February 3, 2026, https://sabrinaramonov.notion.site/FREE-AI-Prompts-Library-Sabrina-Ramonov-6ac894954218492d9fc9e1f7f90abc6c
18. My AGENTS.md for MDFlow - GitHub Gist, accessed February 3, 2026, https://gist.github.com/intellectronica/b89c75f488b68275666ecf9873387d87